ISO/IEC JTC1 SC22 WG21 P3247R1
Jens Maurer <Jens.Maurer@gmx.net>
Target audience: LEWG, CWG
2024-05-19

P3247R1: Deprecate the notion of trivial types

Introduction

The core language defines "trivial types", and in particular, trivial class types, even though that concept is used mainly for the definition of the library type trait `is_trivial`. This proposal deprecates the type trait `is_trivial` and moves the definition of "trivial type" next to the new location of the specification of the type trait.

This proposal follows the footsteps of "POD", deprecated with P0767R1 (November, 2017).

This proposal partially addresses core issue 1808.

History

- R1: Adjust two more uses of "trivial class" in the standard library; add forgotten change to [meta.type.synop].

Wording changes

Remove from 6.9.1 [basic.types.general] paragraph 9:

> [...] ~~Scalar types, trivial class types (11.2 [class.prop]), arrays of such types, and cv-qualified versions of these types are collectively called trivial types.~~ [...]

Change in 11.2 [class.prop] paragraph 2:

> ~~A trivial class is a class that is trivially copyable and has one or more eligible default constructors (11.4.5.2), all of which are trivial. [Note 1: In particular, a trivially copyable or trivial class does not have virtual functions or virtual base classes. — end note]~~

Change in 11.2 [class.prop] paragraph 7:

> [Example 2 :

```
struct N {   // neither ~~trivial~~ **trivially copyable** nor standard-layout
  int i;
  int j;
  virtual ~N();
};

struct T {  // ~~trivial~~ **trivially copyable** but not standard-layout
  int i;
private:
  int j;
};

struct SL {  // standard-layout but not ~~trivial~~ **trivially copyable**
  int i;
  int j;
```

```
  ~SL();
};

struct POD {  // both trivial trivially copyable and standard-layout
  int i;
  int j;
};
```

— end example]

Change in 11.9.5 [class.cdtor] paragraph 1:

```
extern X xobj;
int* p3 = &xobj.i;    // OK, X is a trivial class has no non-trivial constructors
X xobj;
```

Change in 17.2.4 [support.types.layout] paragraph 5:

The type `max_align_t` is a ~~trivial~~ **trivially copyable, default-constructible** standard-layout type **whose only default constructor is trivial and** whose alignment requirement is at least as great as that of every scalar type, and whose alignment requirement is supported in every context (6.7.6).

Change in 23.1 [strings.general] paragraph 1:

This Clause describes components for manipulating sequences of any non-array ~~trivial~~ **trivially copyable** standard-layout (6.8.1) type **where default-initialization of an object of such a type invokes a trivial default constructor, if any**. Such types are called char-like types, and objects of char-like types are called char-like objects or simply characters.

Remove from 21.3.3 [meta.type.synop]:

```
template<class T> struct is_trivial;
```

...

```
template<class T>
  constexpr bool is_trivial_v = is_trivial<T>::value;
```

Remove from 23.3.5.4 [meta.unary.prop]:

`template<class T>` `struct is_trivial;`	T is a trivial type (6.9.1 [basic.types.general])	`remove_all_extents_t<T>` shall be a complete type or *cv* `void`.

Change in 24.7.3.4.4 [mdspan.layout.policy.overview] paragraphs 1 and 2:

Each of `layout_left`, `layout_right`, and `layout_stride` **, as well as each specialization of `layout_left_padded` and `layout_right_padded`,** meets the layout mapping policy requirements and is a ~~trivial~~ **trivially copyable** type. **Furthermore, default-initialization of an object of such a type invokes a trivial default constructor**.

~~Each specialization of layout_left_padded and layout_right_padded meets the layout mapping policy requirements and is a trivial type.~~

Change in D.14 [depr.meta.types]:

```
namespace std {
  template<class T> struct is_trivial;
  template<class T> constexpr bool is_trivial_v = is_trivial<T>::value;
  template<class T> struct is_pod;
```

```
  template<class T> constexpr bool is_pod_v = is_pod<T>::value;
...
}
```

The behavior of a program...

A *trivial class* is a class that is trivially copyable and where default-initialization of an object of such a class type invokes a trivial default constructor (11.4.5.2 [class.default.ctor]). [Note: In particular, a trivial class does not have virtual functions or virtual base classes. — end note] A *trivial type* is a scalar type, a trivial class, an array of such a type, or a cv-qualified version of one of these types.

A POD class is a class that ...

```
  template<class T> struct is_trivial;
```

Preconditions: `remove_all_extents_t<T>` shall be a complete type or *cv* `void`.

Remarks: `is_trivial<T>` is a Cpp17UnaryTypeTrait (21.3.2 [meta.rqmts]) with a base characteristic of `true_type` if `T` is a trivial type, and `false_type` otherwise.

[Note: It is unspecified whether a closure type (7.5.5.2 [expr.prim.lambda.closure]) is a trivial type. — end note]

```
  template<class T> struct is_pod;
```